L14: Neurons, Neural Networks and Perceptrons

C. V. Jawahar

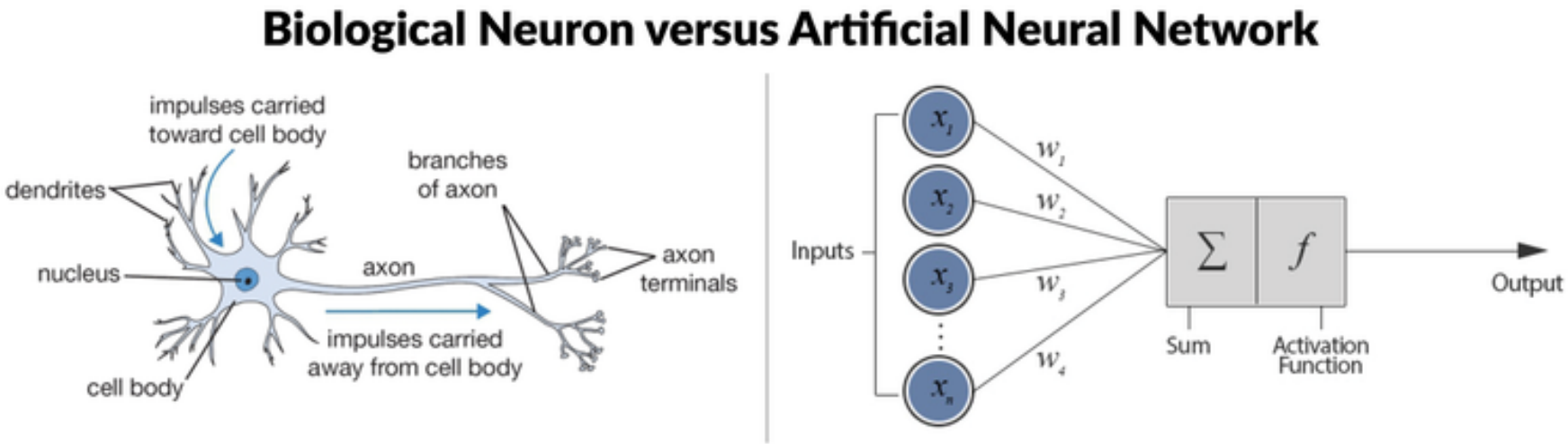
IIIT Hyderabad

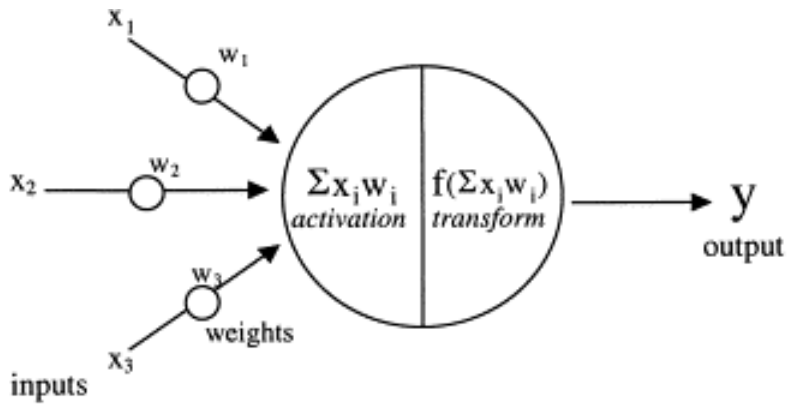
February 25, 2026

Recap:

- Problems of interest:
  - Learn a function $y = f(\mathbf{W}, \mathbf{x})$ from the data: (a) Classification (b) Regression
  - Learn Feature Transformations $\mathbf{x}' = \mathbf{W}\mathbf{x}$ or $\mathbf{x}' = f(\mathbf{W}, \mathbf{x})$
    - (a) Feature Normalization (b) PCA/Dimensionality Reduction
  - Low rank data matrix and SVD; PCA and Eigen Faces
  - Supervised Learning: (i) Train, Val and Test, (ii) Cross Validation and Overfitting (iii) Loss functions and Regularization.
  - Optimization (i) Convex and non-convex (ii) Gradient Descent, Eigen Vector based, EM and MLE
- Algorithms/Approaches:
  - (i) Nearest Neighbour Algorithm (ii) Linear Classification: $sign(\mathbf{w}^T\mathbf{x})$ (iii) Bayesian view; Decide as ω_1 if $P(\omega_1|\mathbf{x}) \geq P(\omega_2|\mathbf{x})$ else ω_2.
  - Linear Regression: (a) closed form and (b) GD; Ridge and Lasso
  - K Means; GMM, Naive Bayes, Logistic Regression, SVMs and Kernels
- Neural Networks:
  - Neuron Models, Perceptrons

Neurons and Neural Networks





Introduction to Neural Networks

- Neural networks can be robustly used for approximating real-valued, discrete-valued and vector-valued target functions.
- Neural networks can store, apply and acquire knowledge. It can *learn* from observations. It can *adapt* to the situations.
- The capability of human brain to recognise events and objects can be learned by the neural networks.
- Human brain is a complex nonlinear parallel computer with approximately 10 billion neurons and 60 trillion synapses or connections.

When to use Neural networks?

- Instances are represented by many attribute-value pair.
- The target function output may be discrete values, real-valued or a vector of several real or discrete valued.
- The training examples may contain errors. Noisy Data.
- Long training time and Fast evaluation .
- The ability of humans to understand the learned target function is not important.

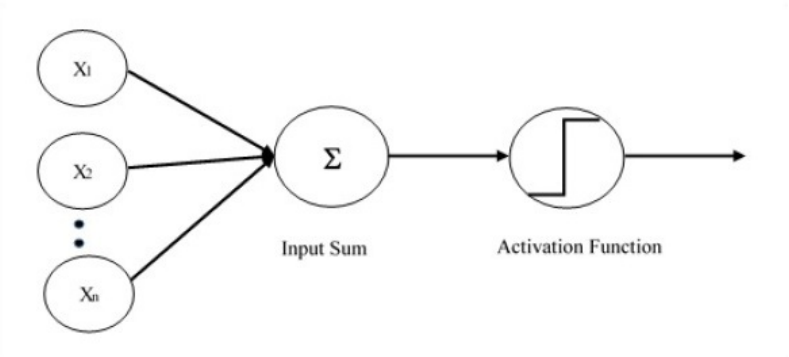
NN and Conventional Computations:

- Sequential Vs. Parallel
- CPUs are complex architectures, neurons are simple structures. Neurons are typically 5 to 6 order slower than silicon gates.
- Number of instructions per sec. Vs number of interconnections.
- Formulate mathematical model and validate with real data. NN identifies the model directly from the data.

Neuron Models

- A neural network consists of a set of neurons arranged in a specific manner. Based on the mathematical model of the neuron, their organisation and the learning algorithm employed, the network becomes capable of solving various pattern recognition tasks.
- A neuron typically takes inputs from various neurons and output its activation state. Inputs are usually weighed and added.
- Let w_{kj} is the weight of connection between jth and kth neurons.
- Adder within the neuron sums the weighted inputs. Output is a nonlinear activation function of this weighted sum. Activation function also limits the value of the output signal to some finite value

Single Layer Perceptron



$$\mathbf{x} = [+1, x_1, x_2, \ldots, x_m]$$

$$\mathbf{w} = [b, w_1, w_2, \ldots, w_m]$$

t be the desired/target response, o be the output (and b be the bias).
$v = \mathbf{w}^T\mathbf{x}$ ϕ: sgn(v) $= +1$ if $v > 0$ and -1 if $v < 0$

Perceptron rule: If samples are incorrectly classified, $w^{\mathbf{k+1}} \leftarrow w^k + \eta t_i \mathbf{x}_i$

Perceptron Learning Algorithm

Problem: Classify samples into two classes as $sign(\mathbf{w}^T\mathbf{x})$

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

Where

- t = c(x) is target value.
- o is perceptron output.
- η is small constant called *learning rate*

Initialize all weights w_i to random values While not all examples are correctly predicted

- DO FOR each training example x in D
  - Compute current output o(x)
  - FOR each $i = 1$ to n
    $w_i < -w_i + \eta(t - o)x_i$

Perceptron: Convergence

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output
- η is small constant (e.g., 0.1) called <u>learning rate</u>

Perceptron training rule

Can prove it will converge

- If training data is linearly separable
- and η sufficiently small

Gradient Descent and Delta Rule

Consider a neuron with an **unthresholded** output

$$o(x) = \mathbf{w} \cdot x$$

Training error can be visualised as

$$E(\mathbf{w}) = \frac{1}{2}\sum_d (t_d - o_d)^2 = \frac{1}{2}\sum_d (t_d - \sum_i w_i x_i)^2$$

What is the nature of this error surface?
Gradient

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right]$$

Problem

Look at the objective/error function with "unthresholded" output:

$$E(\mathbf{w}) = \frac{1}{2}\sum_d (t_d - o_d)^2 = \frac{1}{2}\sum_d (t_d - \sum_i w_i x_i)^2$$

Do you think it is convex and has only one minima? Can you plot this function for a simple case of $d = 1$? (choose your favorite t and o.)

Training rule

$$\Delta \vec{w} = -\eta \nabla E[\vec{w}]$$

i.e.,

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_d (t_d - o_d)^2 = \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
&= \sum_d (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x_d}) \\
\frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d)(-x_{i,d})
\end{aligned}$$

Gradient-Descent

($training_examples, \eta$)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., .05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
  - Input the instance $\vec{x}$ to the unit and compute the output o
  - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i$$

Comments

Perceptron training rule guaranteed to succeed if

- Training examples are linearly separable
- Sufficiently small learning rate η

Linear unit training rule uses gradient descent

- Guaranteed to converge to hypothesis with minimum squared error
- Given sufficiently small learning rate η
- Even when training data contains noise
- Even when training data not separable by H

Incremental (Stochastic) Gradient Descent

Batch mode Gradient Descent:
Do until satisfied

- Compute the gradient $\nabla E_D[\vec{w}]$
- $\vec{w} \leftarrow \vec{w} - \eta \nabla E_D[\vec{w}]$

Incremental mode Gradient Descent:
Do until satisfied

- For each training example d in D
  1. Compute the gradient $\nabla E_d[\vec{w}]$
  2. $\vec{w} \leftarrow \vec{w} - \eta \nabla E_d[\vec{w}]$

$$E_D[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

$$E_d[\vec{w}] \equiv \frac{1}{2} (t_d - o_d)^2$$

<u>Incremental Gradient Descent</u> can approximate <u>Batch Gradient Descent</u> arbitrarily closely if η made small enough

Understanding perceptrons as GD?

$$J' = \sum_{i=1}^{N} (t_i - o_i)^2$$

with $o_i = sign(\mathbf{w}^T \mathbf{x}_i)$? There is a problem. With small changes in the $\mathbf{w}$ or the line, the J is not changing. Let us now define:

$$J = \sum_{i=1}^{N} (t_i - o_i)^2 (-\mathbf{w}^T \mathbf{x}_i)$$

This additional terms pulls (or pushes) *proportionally*.
Let us now rewrite this J as sum of two parts one over $\mathcal{E}$ and the other on not in $\mathcal{E}$.

$$J = J_1 + J_2 = \sum_{\mathbf{x_i} \in \mathcal{E}} (t_i - o_i)^2 (-\mathbf{w}^T \mathbf{x}_i) + \sum_{\mathbf{x}_i not \in \mathcal{E}} (t_i - o_i)^2 (-\mathbf{w}^T \mathbf{x}_i)$$

Understanding perceptrons as GD?

$$J = \sum_{i=1}^{N} (t_i - o_i)^2 (-\mathbf{w}^T \mathbf{x}_i)$$

This additional terms pulls (or pushes) *proportionally*.
Let us now rewrite this J as sum of two parts one over $\mathcal{E}$ and the other on not in $\mathcal{E}$.

$$J = J_1 + J_2 = \sum_{\mathbf{x_i} \in \mathcal{E}} (t_i - o_i)^2 (-\mathbf{w}^T \mathbf{x}_i) + \sum_{\mathbf{x}_i not \in \mathcal{E}} (t_i - o_i)^2 (-\mathbf{w}^T \mathbf{x}_i)$$

$$J = J_1 + J_2 = \sum_{\mathbf{x_i} \in \mathcal{E}} (2 \cdot t_i)^2 (-\mathbf{w}^T \mathbf{x}_i) + \sum_{\mathbf{x}_i not \in \mathcal{E}} 0 \times (\mathbf{w}^T \mathbf{x}_i)$$

We know that J_2 is zero. When $\mathbf{x}_i \in \mathcal{E}$, $(t_i - o_i)$ is $2t_i$ i.e., $2y_i$.

Are the perceptrons doing GD?

- We know either $(t_i - o_i)$ is zero Or we know that $(t_i - o_i)$ is $2t_i$
- And $\frac{\partial(-\mathbf{w}^T\mathbf{x}_i)}{\partial \mathbf{w}} = -\mathbf{x_i}$.

$$\nabla J = -2\sum_{i=1}^{N}(t_i - o_i)\mathbf{x}_i$$

This leads to: $\mathbf{w}^{k+1} \leftarrow \mathbf{w}^k - \eta\nabla J$ as:

$$\mathbf{w}^{k+1} \leftarrow \mathbf{w}^k - \eta\sum_{i=1}^{N}(t_i - o_i)(-\mathbf{x}_i)$$

$$\mathbf{w}^{k+1} \leftarrow \mathbf{w}^k + \eta\sum_{\mathbf{x}\in\mathcal{E}} 2.t_i\mathbf{x}_i$$

$$\mathbf{w}^{k+1} \leftarrow \mathbf{w}^k + \eta'\sum_{\mathbf{x}_i\in\mathcal{E}} y_i\mathbf{x}_i$$

(with changes in scale for learning rate η)

Perceptron Convergence

- **Claim 1** If there exist a set of weights that are consistent with the data, the perceptron algorithm will converge.
- **Claim 2**If the training data is not Linearly Separable, the perceptron algorithm will eventually repeat the same set of weights and thereby enter an infinite loop.
- **Claim 3** If the training data is linearly separable, algorithm will converge in a maximum of N steps. Find N.
- **Claim 4** Every boolean function can be represented by some network of perceptrons only two levels deep.

Read and understand the proofs for the above claims[1]

[1] Page 229 of Duda, Hart and Stork: https://cds.cern.ch/record/683166/files/0471056693_TOC.pdf

Perceptron: Problem

Consider two samples from A: $[1, 2]^T, [4, 2]^T$ and one sample from B: $[3, 0]^T$. Let us start with a $\mathbf{w} = [1, 1, 0]^T$.

- Show one iteration of perceptron algorithm with $\eta = 0.1$.
- Iterate until convergence

Perceptron: Problem

With ± 1 logic, design a SLP that can do the job of *AND*

Perceptron: Problem

Can we solve *EXOR* problem with SLP? How do we solve it with a simple kernel based expansion of inputs?

Mid Exam

- Q1
- Q2
- Q3